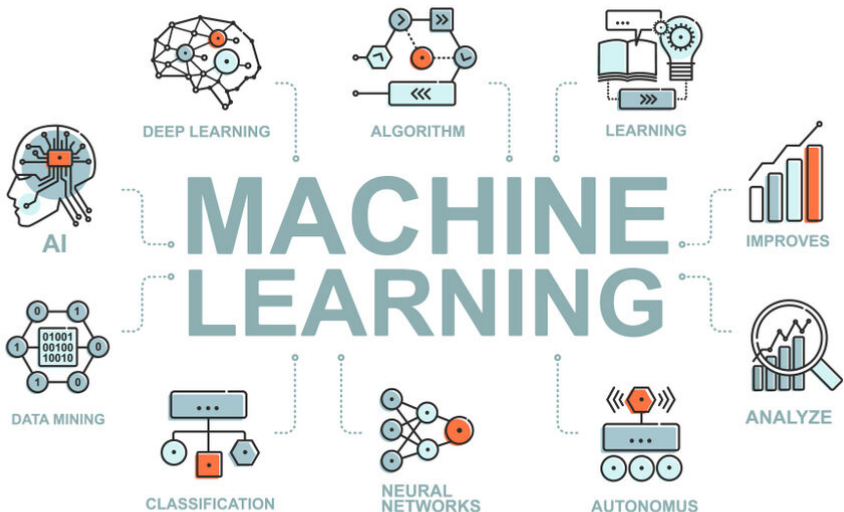


Machine Learning for AI Engineers

The terms and algorithms behind every model you'll ever fine-tune, evaluate, or ship - explained for people who mostly work with LLMs.



AI Coach John



Follow

↻ Repost

You're building on top of this, even **with LLMs**

Skip the fundamentals and here's what goes wrong:

- ✗ You fine-tune a model and have no idea why it got **worse** instead of better.
- ✗ You pick an evaluation metric that actively **hides** the real problem.
- ✗ You read a suspiciously perfect score as a **win** instead of a red flag.
- ✗ You tune by **guessing** rather than reasoning about what's happening.

WHAT'S INSIDE

- 01 Where "I just call an API" breaks down
- 02 The 3 types of machine learning
- 03 11 essential algorithms - what, why, and the tradeoffs
- 04 A 50-term glossary every AI engineer should know



"I don't need this, I just call an API" - here's where that **breaks down**

THE SETUP

A team fine-tunes a model to sort support tickets by urgency. It scores **99% accuracy** in testing. They ship it.

WHAT ACTUALLY HAPPENED

- They evaluated on the same data they fine-tuned on - a basic **train/test split** mistake.
- The model never learned what makes a ticket urgent. It memorised the examples it saw - **overfitting**.
- On real tickets, accuracy collapsed to **~60%**. Cue the rollback, the postmortem, and a team that no longer trusts AI features.

THE FIX

A properly held-out test set, and the instinct to treat a perfect score as a **red flag**, not a result. Neither of these is "old ML" - both apply directly to fine-tuning and shipping LLM systems.



Every algorithm ahead is one of these three

Supervised

Learns from labelled answers

e.g. Predicting a price, Spam detection

Unsupervised

Finds patterns, no labels

e.g. Customer segments, Topic modeling



Reinforcement

Learns from reward and penalty

e.g. Game-playing agents, RLHF for LLMs

WHERE LLMS SIT

Almost everything in this carousel is supervised or unsupervised. Reinforcement learning is the third branch - and it's the one behind RLHF and today's agentic systems.



AI Coach John



PROITBRIDGE
Drive For Better Future

Follow

 Repost

Predicting a continuous number from a straight-line relationship

WHAT IT IS

Fits a straight line through your data points to predict a continuous value.

EXAMPLE

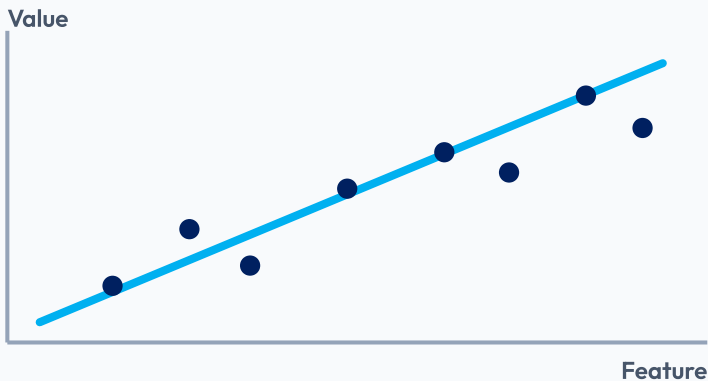
Estimating house price from square footage.

WHY WE USE IT

Simple, fast, and fully interpretable - you can read off exactly how much each feature moves the prediction.

TRADEOFFS

Only captures straight-line relationships. Falls apart when the real pattern curves.



Predicting a category from a probability curve

WHAT IT IS

Despite the name, this classifies. It outputs a probability between 0 and 1, then a threshold decides the class.

EXAMPLE

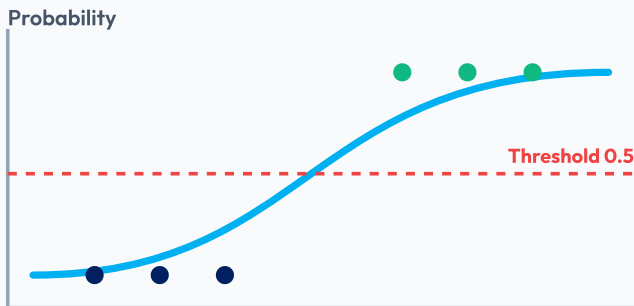
Is this email spam, or not?

WHY WE USE IT

Fast, interpretable, and a strong baseline to beat before you reach for anything heavier.

TRADEOFFS

Assumes a fairly simple decision boundary. Struggles when classes separate in complex ways.



A flowchart of yes / no questions leading to a prediction

WHAT IT IS

Splits the data again and again on feature thresholds, forming a tree that ends in a prediction.

EXAMPLE

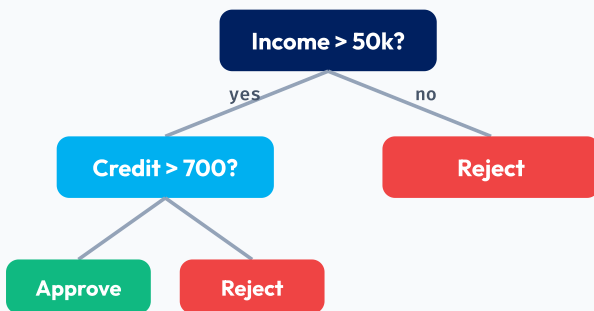
Income above 50k? Then credit score above 700? Then approve the loan.

WHY WE USE IT

Completely readable - you can trace the exact logic behind any single decision.

TRADEOFFS

Overfits easily on its own, and a small change in the data can produce a very different tree.



Many decision trees **voting together**

WHAT IT IS

Trains lots of trees on random slices of the data and features, then takes the majority vote.

EXAMPLE

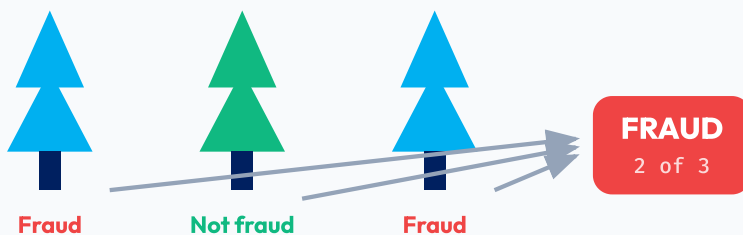
Fraud detection across dozens of transaction signals.

WHY WE USE IT

Far more robust than a single tree - cuts overfitting sharply while staying quick to train.

TRADEOFFS

Harder to interpret than one tree, heavier in memory, and still weak on very sparse high-dimensional data.



All trees vote at once, majority wins



Trees built one after another, each fixing the last one's mistakes

WHAT IT IS

Builds trees in sequence, where every new tree is trained on what the previous ones got wrong.

EXAMPLE

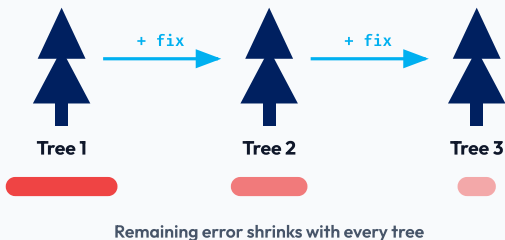
The winning approach in most tabular Kaggle competitions - credit scoring, demand forecasting.

WHY WE USE IT

Usually the strongest performer on structured data, regularly beating deep learning there.

TRADEOFFS

Overfits if you don't tune it carefully, trains slower than a forest, and has more knobs to get wrong.



Classify a point by **what's nearby**

WHAT IT IS

Looks at the k closest labelled points and takes a majority vote (or an average, for numbers).

EXAMPLE

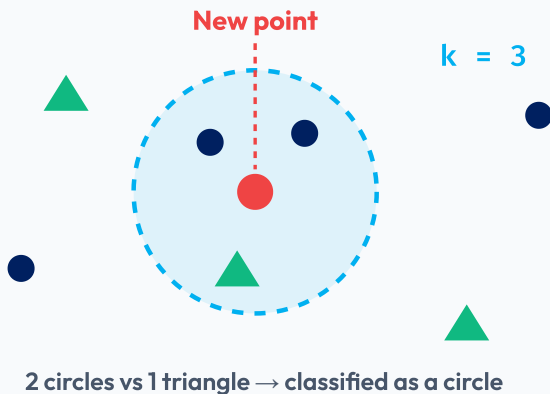
Recommending similar products based on purchase history.

WHY WE USE IT

About as easy as machine learning gets, and there's no training phase at all.

TRADEOFFS

Slow to predict on big datasets, and it's thrown off badly by irrelevant or unscaled features.



Grouping data into k clusters, with no labels needed

WHAT IT IS

Sorts data points into k groups by similarity, without ever being told the right answer.

EXAMPLE

Segmenting customers for marketing.

WHY WE USE IT

Quick and effective for surfacing natural groupings in unlabelled data.

TRADEOFFS

You have to pick k up front, results shift with the starting positions, and it assumes round, evenly sized clusters.



x marks each cluster centre - here, $k = 3$



Finding the widest possible gap between classes

WHAT IT IS

Draws the boundary that maximises the margin between two classes, and uses kernels when the split isn't a straight line.

EXAMPLE

Text classification, or image classification on smaller datasets.

WHY WE USE IT

Holds up well with lots of features and not much data - a common combination in text.

TRADEOFFS

Doesn't scale to very large datasets, and picking the right kernel is fiddly.



Fast, probability-based classification

WHAT IT IS

Uses Bayes' theorem to score each class, assuming every feature is independent - the “naive” part.

EXAMPLE

Spam filtering and sentiment classification.

WHY WE USE IT

Trains and predicts almost instantly, and works surprisingly well on text despite that assumption.

TRADEOFFS

Features are rarely independent in the real world, which caps its accuracy on harder problems.

$$P(A|B) = P(B|A) \times P(A) \div P(B)$$

Probability of a class, given the evidence

assumes every feature is independent



Layers of connected nodes learning complex patterns

WHAT IT IS

Stacked layers of neurons, each applying weights and an activation, trained by backpropagation to cut loss.

EXAMPLE

Image recognition - and the direct ancestor of every transformer behind today's LLMs.

WHY WE USE IT

Learns extremely complex non-linear patterns, given enough data. All of deep learning sits on this.

TRADEOFFS

Data-hungry, expensive to train, and much harder to explain when it gets something wrong.



Compressing many features into a few, without losing much information

WHAT IT IS

Finds new combined axes – principal components – that capture as much of the variation as possible.

EXAMPLE

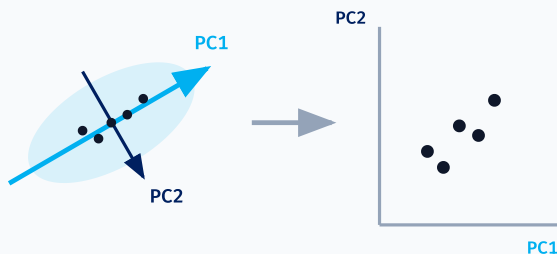
Squashing hundreds of features down to two or three, for plotting or as a preprocessing step.

WHY WE USE IT

Cuts noise and compute, and makes high-dimensional data something you can actually look at.

TRADEOFFS

The new axes aren't interpretable. Nobody can tell you what “component 1” means in business terms.



The vocabulary **everything else assumes**

- **Feature** - an input variable used to make a prediction
- **Label** - the correct answer a model is trained to predict
- **Train / validation / test split** - three separate sets: to learn on, tune on, and finally judge on
- **Overfitting** - memorising the training data instead of learning the pattern
- **Underfitting** - a model too simple to capture the real pattern at all
- **Loss function** - the formula measuring how wrong the predictions are
- **Gradient descent** - the method that nudges parameters downhill to cut loss
- **Epoch** - one full pass through the entire training set
- **Hyperparameter** - a setting you choose before training, like learning rate
- **Bias-variance tradeoff** - the tension between too simple and too sensitive



The building blocks of every modern model

- **Neuron / activation function** - a unit turning weighted inputs into a signal, e.g. ReLU
- **Backpropagation** - the algorithm working out how to adjust every weight
- **Learning rate** - how big a step each update takes
- **Batch size** - how many examples are processed before weights update
- **Dropout** - randomly switching neurons off during training to curb overfitting
- **Batch normalisation** - normalising layer inputs to stabilise and speed up training
- **CNN** - an architecture built for images and spatial data
- **RNN** - the older sequential architecture, largely replaced by transformers
- **Transformer** - the architecture behind essentially every modern LLM
- **Attention** - lets the model weigh which parts of the input matter most



Terms you'll hit working with LLMs

- **Token** - the basic unit of text a model processes, roughly a word piece
- **Tokenisation** - splitting text into those tokens before the model sees it
- **Embedding** - a numerical vector capturing a piece of text's meaning
- **Context window** - the most tokens a model can hold in mind at once
- **Temperature** - how random versus deterministic the generation is
- **Top-p / top-k sampling** - which candidate tokens stay in play during generation
- **Prompt engineering** - shaping the input to get more reliable output
- **Fine-tuning** - further training a pretrained model on your own data
- **RLHF** - training on human preference signals to align behaviour
- **Hallucination** - fluent, confident output that simply isn't true



How models actually **get built**

- **Regularisation (L1 / L2)** - penalising complexity to hold overfitting back
- **Cross-validation** - re-splitting the data repeatedly for a more honest score
- **Learning rate schedule** - a plan for changing the learning rate as training runs
- **Transfer learning** - reusing a model trained elsewhere as your starting point
- **Data augmentation** - growing a training set with modified copies of what you have
- **Early stopping** - halting once validation performance stops improving
- **Optimiser (Adam, SGD)** - the specific algorithm updating the weights
- **Vanishing / exploding gradient** - gradients too small or too large to learn from
- **Quantisation** - lowering weight precision to shrink the model and speed it up
- **LoRA / PEFT** - fine-tuning that adapts a model without retraining every weight



Measuring it, then keeping it alive

- **Precision** - of everything flagged positive, how much really was
- **Recall** - of everything actually positive, how much you caught
- **F1 score** - the harmonic mean of the two, when you need both
- **Confusion matrix** - right versus wrong predictions, broken down by class
- **ROC-AUC** - how well a classifier separates classes across all thresholds
- **Perplexity** - how surprised a language model is by text - lower is better
- **Latency vs throughput** - how fast one request finishes vs how many you can serve
- **Inference** - running a trained model to get predictions
- **Model drift** - performance decaying as the real world moves on
- **MLOps** - the tooling and practice of keeping models running in production



Which one should you actually reach for?

YOUR SITUATION	START WITH
Predicting a number, need interpretability	Linear Regression
Predicting a category, want a strong baseline	Logistic Regression
Need a fully explainable decision path	Decision Tree
Tabular data, want accuracy above all	Gradient Boosting
No labels, want to find groups	K-Means
Small dataset, many features (text)	SVM or Naive Bayes
Complex patterns, plenty of data	Neural Network
Too many features to handle or plot	PCA



Same concepts, **bigger scale**

- **Loss and gradient descent** - exactly what trains an LLM, just across billions of parameters.
- **Overfitting** - why a model can parrot its training data instead of generalising. A live risk in fine-tuning.
- **Bias-variance tradeoff** - why a heavily fine-tuned model loses general capability.
- **Neural networks** - the direct architectural ancestor of the transformer.
- **Precision, recall, F1** - still how you evaluate any classifier you build on top of an LLM.

THE POINT

None of this is legacy knowledge. It's the layer underneath the API call.



Mistakes even experienced engineers make

- ⚠ **Evaluating on data the model already saw.** It gives you confidence you haven't earned.
- ⚠ **Ignoring feature scaling.** KNN and SVM break when age sits next to income unscaled.
- ⚠ **Reaching for complexity first.** A tuned Logistic Regression often beats a rushed neural network on tabular data.
- ⚠ **Guessing k in K-Means.** Test a few values with the elbow method instead.
- ⚠ **Trusting a black box on faith.** Sanity-check predictions against what you know about the domain.

RULE OF THUMB

Every one of these is caught by the same habit: be suspicious of your own good results.

